SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Introduction

Error analysis in software testing identifies weaknesses in test design, coverage assumptions, and the reasoning used to select test cases. A suite may execute successfully yet miss defects if it combines valid and invalid inputs, ignores boundary values, omits decision branches, or relies only on random or ad-hoc testing. This assignment evaluates ten scenarios and proposes corrections using equivalence class partitioning, boundary value analysis, control-flow graphs, branch and path coverage, condition coverage, decision tables, loop testing, state-transition testing, and error guessing.

Where the assignment does not provide a complete condition or system-specific limits, the examples below state reasonable assumptions. Final test values should be confirmed against the actual requirements. Question 6 is incomplete in the uploaded file, ending at “if(A && B”; its answer therefore explains the general method using that visible expression.

1. Errors in Equivalence Partitioning and Boundary Value Analysis for Online Shopping

Understanding the problem: The tester combines valid and invalid inputs in one equivalence class and ignores boundary values. Equivalence Class Partitioning (ECP) is useful only when values in a class are expected to behave similarly. A mixed class cannot be represented reliably by a single test.

Errors identified: (1) valid and invalid values are mixed; (2) classes may be incomplete; (3) empty, malformed, and out-of-range values may be omitted; (4) boundary values are not selected; (5) expected outcomes are not documented.

Root causes: The tester may misunderstand ECP, focus only on normal inputs, or fail to check requirements for quantity limits, stock availability, discount thresholds, and accepted payment values. Without explicit rules, the test data becomes arbitrary.

Corrected partitions: For shopping quantity, define valid integer quantity from 1 through the permitted maximum; zero; negative quantity; non-numeric input; fractional quantity if only whole units are allowed; and quantity above the maximum. For a price filter, include a valid non-negative range, negative price, malformed price, minimum greater than maximum, and range outside the catalogue. For coupons, include valid active coupon, expired coupon, unknown coupon, empty coupon, and coupon not applicable to the selected product.

Boundary value analysis: If the maximum quantity is M and the minimum is 1, test 0, 1, 2, M−1, M, and M+1. For a discount threshold P, test P−1, P, and P+1 in the smallest supported currency unit. Use actual configured limits in the final test sheet.

Corrected cases: (1) quantity=1: accept if stock exists; (2) quantity=0: validation error; (3) quantity=−1: validation error; (4) quantity=M: accept if within stock and policy; (5) quantity=M+1: reject or constrain according to requirements; (6) quantity="abc": type validation error; (7) fractional quantity where whole units are required: reject; (8) valid quantity exceeding stock: stock error; (9) price filter at exact endpoints: follow inclusive/exclusive rules; (10) expired coupon: reject and do not apply discount.

Justification: Correct partitions cover distinct behaviors, while boundary tests target off-by-one errors. Separating stock, price, and coupon rules makes failures easier to diagnose and increases defect detection compared with a few ordinary-value tests.

2. Evaluating Random Testing in a Banking Application

Evaluation: Random testing generates inputs without a structured relationship to requirements or risk. It can discover unexpected failures, but it does not guarantee that transaction limits, exact boundaries, or critical combinations are tested. The claim of high coverage is unsupported unless coverage is defined and measured.

Errors in strategy: The tester treats the number of random tests as proof of coverage, does not partition valid and invalid inputs, ignores boundary values, fails to prioritize high-risk transactions, and may not verify balances and transaction records after failures or retries.

Root causes: Random generators often select typical values rather than exact endpoints. Rare cases—such as balance equal to withdrawal amount, a daily limit just reached, or timeout after authorization—may be missed. Without an expected-result oracle, random inputs alone cannot establish correctness.

Correct techniques: Equivalence Class Partitioning groups inputs by expected behavior. Boundary Value Analysis tests values at and around limits. Decision Table Testing checks combinations such as valid authentication, active account, sufficient balance, and limit not exceeded. State-Transition Testing checks states such as initiated, pending, completed, failed, and reversed. Random testing should supplement these methods.

Example tests: (1) valid withdrawal below limit and within balance: one debit; (2) zero amount: reject; (3) negative amount: reject; (4) one unit below minimum: reject; (5) amount at minimum: follow policy; (6) amount at maximum: accept if inclusive; (7) one unit above maximum: reject; (8) amount equal to balance: verify minimum-balance rule; (9) one unit above balance: insufficient funds; (10) daily total below, equal to, and above limit; (11) timeout after request: reconcile before retry; (12) duplicate request ID: no double debit; (13) simultaneous withdrawals: prevent overspending; (14) invalid authentication: no transaction.

Adequacy: Use a requirements-to-test matrix and record class, boundary, decision-combination, and state-transition coverage. Random tests can use a fixed seed for reproducibility. Check actual balances and transaction records against expected results after every test. This is more reliable than unstructured random testing alone.

3. Correcting an Incomplete Control Flow Graph and Path Testing

A control flow graph (CFG) represents program execution as nodes and directed edges. Nodes represent statements or basic blocks; edges represent possible transfers of control. Omitting decision nodes or loops means the graph cannot represent all execution behavior.

Errors and causes: The developer may represent an if statement as straight-line logic, omit false edges, miss loop entry/exit edges, ignore early returns or exceptions, or fail to connect paths after a decision. This often happens when the graph is drawn from a high-level summary rather than the actual code.

Illustrative corrected CFG: Start → Validate order → [Valid?]. No → validation error → End. Yes → initialize item index → [More items?]. Yes → check stock → [Available?]. No → stock error → End. Yes → advance index → return to [More items?]. No → process payment → [Payment accepted?]. No → payment error → End. Yes → confirm order → End. The actual CFG must include all decisions and loops present in the real source.

Independent paths: Path 1: invalid order → validation error. Path 2: valid order → no items remain → payment accepted, if empty orders are allowed. Path 3: valid order → available item(s) → loop exits → payment accepted. Path 4: valid order → item unavailable → stock error. Path 5: all items available → payment declined. Path 6: multiple available items → loop repeats several times → payment accepted.

Corrected tests: Test invalid and valid orders, available and unavailable stock, accepted and declined payment, and zero, one, and multiple loop iterations where permitted. Use branch coverage for true and false outcomes and condition coverage for compound expressions. A coverage tool can reveal unvisited edges.

Justification: A complete CFG exposes error paths hidden by a straight-line diagram. Independent path testing exercises distinct decisions and loop outcomes. Full path coverage may be impossible for unbounded loops, so use a practical basis of independent paths and representative loop counts.

4. Why 100% Statement Coverage Misses Conditional Defects

Statement coverage means every executable statement has run at least once. It does not mean every decision outcome has been exercised. A test may execute a calculation statement while never testing a false branch or an important Boolean combination.

Example: if (memberIsPremium && couponIsValid), apply a special discount; otherwise apply normal pricing. A test where both conditions are true executes the discount statement, but it does not show behavior when the member is not premium or the coupon is invalid.

Coverage gaps: Statement coverage does not guarantee branch, condition, decision, or path coverage. It may miss incorrect comparison operators, missing else branches, Boolean precedence mistakes, and interactions between conditions. Assertions are also needed to verify that the output is correct.

Recommendations: Use branch coverage to execute true and false outcomes; condition coverage to make each atomic condition true and false; MC/DC for important compound decisions where required; decision tables to cover combinations; and boundary tests for numeric comparisons.

Example tests: (A) premium member and valid coupon; (B) premium member without valid coupon; (C) non-premium member with valid coupon; (D) non-premium member without coupon; (E) values just below, at, and above numeric thresholds; (F) missing or malformed inputs. Expected discount behavior must follow the actual business rules.

Conclusion: Statement coverage is a useful minimum, but branch and condition coverage, decision tables, and path testing provide stronger evidence that conditional alternatives have been checked.

5. Correcting Age Equivalence Classes for the Range 0–100

Error analysis: The tester defines only negative and positive values. Positive values include both valid ages from 1 to 100 and invalid ages above 100. The value 0 is not handled clearly, and non-numeric or fractional input is ignored.

Corrected classes: (1) valid integer age from 0 through 100 inclusive; (2) age below 0; (3) age above 100; (4) non-numeric input; (5) fractional value if only whole years are accepted; (6) missing or empty input. If the field accepts only integer years, unsupported formatting should also be tested.

Boundary value analysis: Test −1, 0, 1, 99, 100, and 101. These values lie outside, at, and inside both boundaries. Also test blank input, text, and a very large numeric value to check safe parsing.

Expected results: −1 → reject; 0 → accept if explicitly permitted; 1 → accept; 99 → accept; 100 → accept; 101 → reject; text such as “twenty” → validation error; 18.5 → follow the specified data type; empty input → required-field error if mandatory.

Justification: The revised classes separate valid and invalid behavior and prevent the broad positive class from hiding the invalid value 101. Boundary tests directly verify the minimum and maximum rules.

6. Condition Coverage for a Complex If Statement

The uploaded assignment text for Question 6 is incomplete: it ends at “A developer claims full Condition Coverage for a complex if(A && B”. The full expression and remainder of the question are not visible. This answer therefore explains the method using the visible illustrative expression A && B.

Condition coverage requires each atomic Boolean condition to evaluate to both true and false at least once. For A && B, the atomic conditions are A and B. Example tests are: Test 1 A=true, B=true → decision true; Test 2 A=true, B=false → decision false; Test 3 A=false, B=true → decision false; Test 4 A=false, B=false → decision false. These four combinations make the behavior straightforward to inspect.

Important limitation: In languages with short-circuit evaluation, B may not actually be evaluated when A is false. Testers must distinguish logical test values from conditions actually evaluated by the implementation. Condition coverage does not always guarantee decision coverage or prove that each condition independently changes the decision outcome.

Recommendation: Confirm the complete expression, list every atomic condition, select the required coverage criterion, and document which tests make each condition true and false. For safety-critical or complex decisions, consider Modified Condition/Decision Coverage (MC/DC) and demonstrate each condition’s independent effect.

7. Correcting an Incomplete Decision Table for a Discount System

Problem: The team omitted combinations of Premium Member and Coupon Applied. A decision table lists condition combinations and the action expected for each rule. Missing combinations mean some price calculations are never tested.

Assumptions: Let C1 = customer is a premium member and C2 = a valid coupon is applied. The exact business rule for combining discounts must be confirmed; the assignment does not specify whether discounts are additive, sequential, capped, or mutually exclusive.

Exhaustive two-condition table: Rule 1: Premium=Yes, valid coupon=Yes → apply the documented combined-discount rule. Rule 2: Premium=Yes, valid coupon=No → premium discount only. Rule 3: Premium=No, valid coupon=Yes → coupon discount only. Rule 4: Premium=No, valid coupon=No → no discount.

Additional tests: expired coupon; coupon restricted to a different product; purchase amount below, at, and above the coupon minimum; repeated coupon submission; and price rounding. Each test should assert the final amount as well as the displayed discount.

Justification: Two binary conditions produce four combinations, all of which can be tested directly. The table makes missing combinations visible and connects every rule to a test. Requirements must define how discounts combine before expected prices can be calculated accurately.

8. Correcting Loop Testing for Zero, Maximum, and Excessive Iterations

Errors: The tester checks only n=1 and n=max. This misses zero iterations, where the loop body may not execute, and n=max+1, which checks enforcement beyond the supported limit. Defects may also occur on the second iteration or during repeated state updates.

Root causes: The tester may assume each iteration behaves identically, overlook initialization and termination conditions, or fail to distinguish valid counts from invalid input. Nested loops require additional attention because inner-loop behavior can depend on the outer iteration.

Corrected simple-loop strategy: For maximum M, test 0, 1, 2, a typical middle value, M−1, M, and M+1 where applicable. Avoid duplicate values when M is small. Verify initialization, entry, exit, counter value, and the effects of each iteration. At M+1, the system should reject or handle the input according to requirements.

Nested loops: Test the innermost loop while outer loops use minimum values, then expand outward one loop at a time. Test zero and one iteration in each loop, typical and maximum counts, and cases where an inner loop runs zero times for one outer iteration and multiple times for another. Full combinations can grow rapidly, so prioritize risk and interactions.

Example: For a batch processor supporting up to M records, test an empty batch, one record, two records, M−1, M, and M+1. Verify every record is processed exactly once, the final record is not skipped, and over-limit input does not corrupt partial results.

Justification: Zero tests the bypass path; one and two iterations test initialization and repeated updates; maximum checks capacity and termination; maximum-plus-one checks limit enforcement. This is more complete than first-and-last-only testing.

9. Error Guessing in a Safety-Critical Braking System

Why error guessing alone is inadequate: Error guessing uses experience to predict likely defects, but it is subjective and may not cover every requirement, boundary, input class, or state transition. In a safety-critical braking system, relying only on ad-hoc tests creates uncertainty because missed defects could affect braking behavior.

Risks and causes: Tests may omit operating limits, rare sensor conditions, mode transitions, or combinations of speed, load, road condition, and sensor status. Tests may lack repeatability, traceability, and measurable coverage criteria.

Apply Boundary Value Analysis: For each specified numeric input—such as speed, brake pressure, sensor range, or temperature—test the minimum, nominal, and maximum values, and values just below and above each boundary where safe and meaningful. Use real engineering limits, not invented values. Verify that out-of-range sensor readings lead to the specified safe response.

Apply State-Transition Testing: Model states such as standby, braking requested, braking active, anti-lock control active, fault detected, and safe/degraded mode only where supported by the real system specification. Test every permitted transition and representative prohibited transitions. Include sensor failure during braking, signal loss and restoration, and recovery from faults according to requirements.

Combined strategy: Link safety requirements to BVA tests, state transitions, expected outputs, and hazard controls. Add branch/condition testing, fault injection, timing tests, and simulation or hardware-in-the-loop tests where appropriate. Error guessing can supplement this structured suite.

Justification: BVA concentrates on thresholds where control decisions change. State-transition testing checks sequences, revealing defects that occur only in particular modes. The combination gives stronger evidence than ad-hoc testing alone, although testing by itself does not prove a safety-critical system safe.

10. Correcting a CFG That Omits Else Branches in Nested If Statements

Error analysis: A CFG that omits else branches fails to represent all possible control transfers. Tests may execute every statement on the successful path and achieve 100% statement coverage while never executing error handlers. Zero branch coverage on error paths shows that important decision outcomes remain untested.

Root causes: The graph may have been drawn from the main success scenario rather than source code. The tester may treat nested decisions as one decision, omit false edges, or ignore early returns and error handlers. Statement coverage may also have been mistaken for branch coverage.

Illustrative corrected CFG: Start → [Request valid?]. No → validation error → End. Yes → [User authorized?]. No → authorization error → End. Yes → [Sufficient funds?]. No → insufficient-funds error → End. Yes → execute transaction → success response → End. Each decision has true and false edges. The actual CFG must reflect the implementation's real merge points, returns, exceptions, and any additional conditions.

Independent paths: Path 1: invalid request → validation error. Path 2: valid request → unauthorized user → authorization error. Path 3: valid and authorized request → insufficient funds → error. Path 4: valid, authorized request → sufficient funds → transaction success.

Corrected tests: (A) malformed request → validation branch; (B) valid request with unauthorized identity → authorization branch; (C) valid and authorized request with insufficient funds → balance-error branch; (D) valid and authorized request with sufficient funds → success branch. Assert both response and system state. Error tests must confirm no transaction occurs and no balance changes.

Coverage and justification: Use branch coverage for both outcomes of every decision, condition coverage for compound expressions, and path testing for independent paths. Compare coverage reports with the CFG and add tests for each feasible unvisited branch. The corrected graph makes error paths explicit and testable.

Overall Conclusion

These scenarios show that effective software testing requires systematic test selection and meaningful coverage measures. Equivalence classes must separate inputs that produce different behavior; boundary value analysis targets values at and around limits. White-box testing requires accurate control-flow graphs and deliberate coverage of branches, conditions, loops, and independent paths. Decision tables expose missing combinations, state-transition tests check sequences, and error guessing is best used as a supplement rather than the sole method for high-risk systems.

Each test case should specify preconditions, input data, expected result, actual result, and pass/fail status. Where the assignment text is incomplete or does not specify exact thresholds, final values and expected outcomes must be confirmed from the complete question and system requirements.

Code of Conduct

I **Sadhana K/192421137** certify that this submission is my work and that I have followed the academic integrity guidelines applicable to this assessment.

Signature: ***Sadhana K***

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand